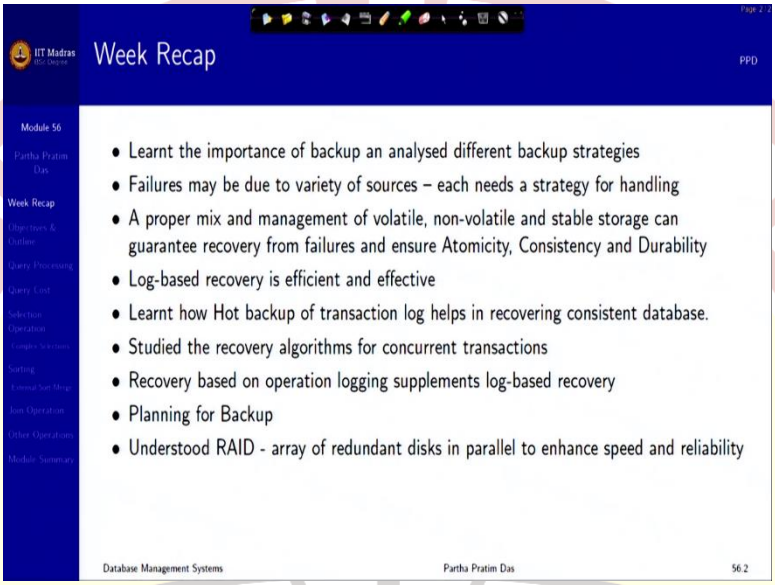
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Query Processing and Optimization/1: Optimization

Welcome to Module 56, week 12 that is the last week of Database Management Systems course in the IIT Madras online BSc programme.

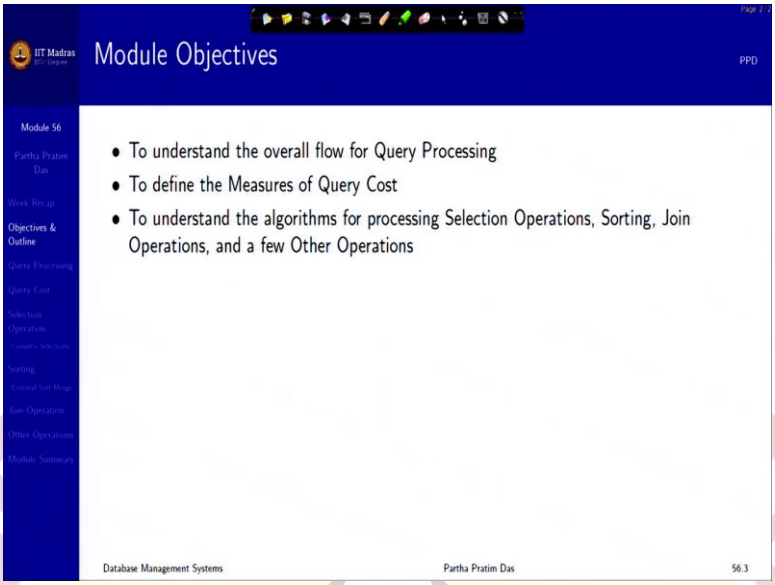
(Refer Slide Time: 00:33)



In the last week, over the five modules we have talked at length about backup and recovery, learn the importance of backup, analyse different strategies, try to understand what are the sources from where failures can happen and how volatile, non-volatile and stable storage can guarantee recovery from failure to ensure atomicity, consistency and durability.

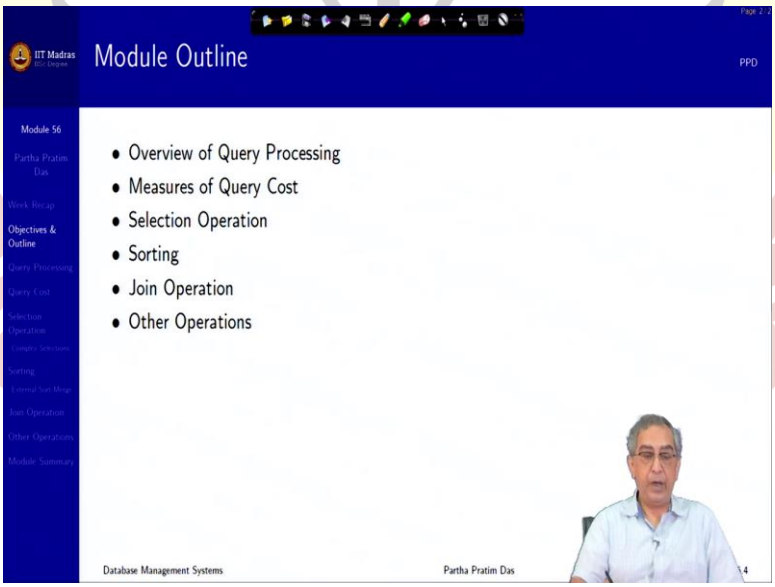
We have learned also about the log-based recovery, its efficiency and effectiveness and how hot backup of transaction log can really help in recovering consistent databases. Studied different recovery algorithms for concurrent transactions, recovery based on operation logging and so on and ended with a plan outline for backups. And in the last module we understood about redundant array of independent disks, their different levels and what are their advantages and otherwise.

(Refer Slide Time: 01:35)



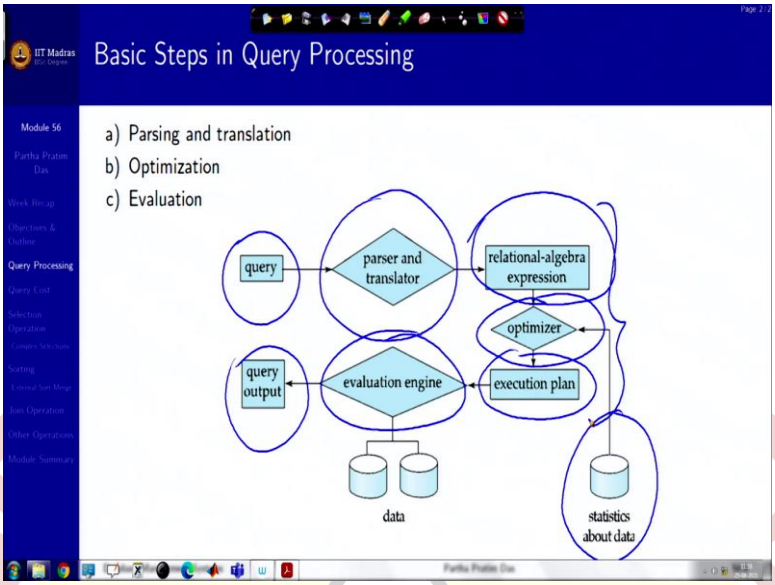
In this module, we will start by understanding the query processing, how does, how does query overall flows in the system, an SQL query, what are the measures of cost and how typically the different operations of selections sorting, join etc, can be done and what could be the ensuing costs in the next two module, we will talk about how to optimise them based on this.

(Refer Slide Time: 02:04)



So, these are the main points we are going to talk about in this module. So first, the overview of query processing.

(Refer Slide Time: 02:14)



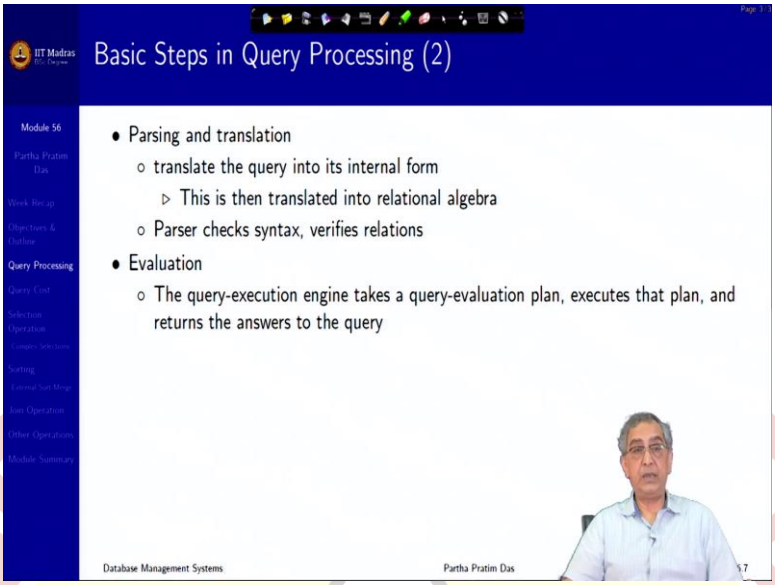
So, you had seen this diagram before and I will again go through that flow here is a query, it is an SQL query. So, you pars, there is a parser and translator, which parses the SQL language like any other programming language and translates it in terms of relational algebra expressions, we had explained at length that the how SQL is equivalent to relational calculus and relational algebra.

So, this relational algebra expression needs to be evaluated in the context of the relations that exist and often the relational expression that is created is not best suited for different, I mean different costs primarily or primary cost is about time, we want it fast and at times we also consider space.

So, through the optimization, we finally come up with an execution plan, which talks about how to go about, what are the different steps to be done in terms of the B plus tree indexes and all that to actually execute the query and the optimization is taken, takes a good guidance from the statistics about data, we talked about how to compute, get statistics regularly collected in the system catalogue.

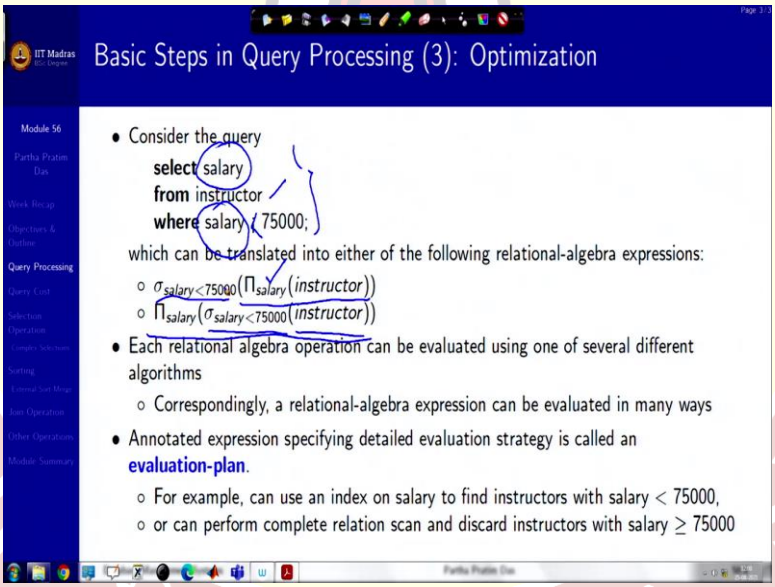
Then based on the execution plan, the exit evaluation engine actually accesses the data and executes every step of the plan to finally get the query output. So, this is the overall flow and this part is what we are going to focus on, we are not going to talk on the parser translator that is once in for all written and that is this is not the course to go deep into that, neither we are going to talk about the actual evaluation engine, but we will talk about what are the relational algebra expressions and how some expressions can be more preferred than others in terms of the costs that may be involved in evaluating it.

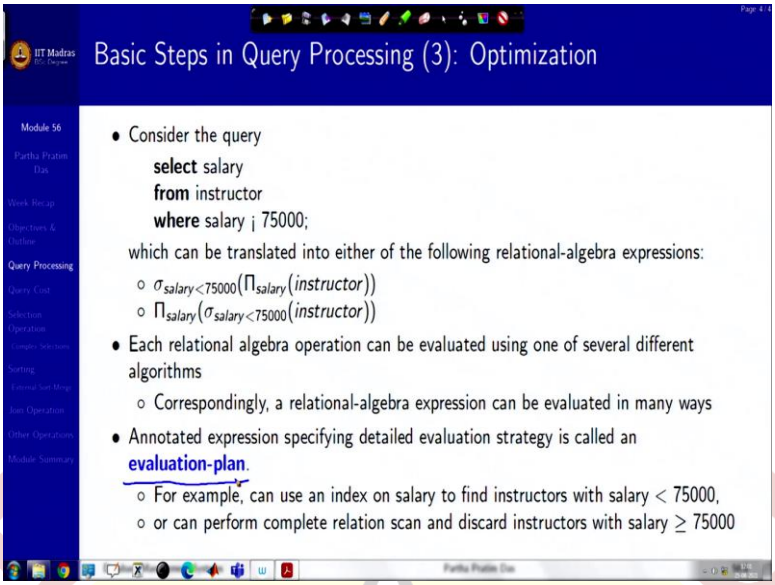
(Refer Slide Time: 04:18)



So, this is the basic steps parsing translation and evaluation to get to a query evaluation plan.

(Refer Slide Time: 04:28)





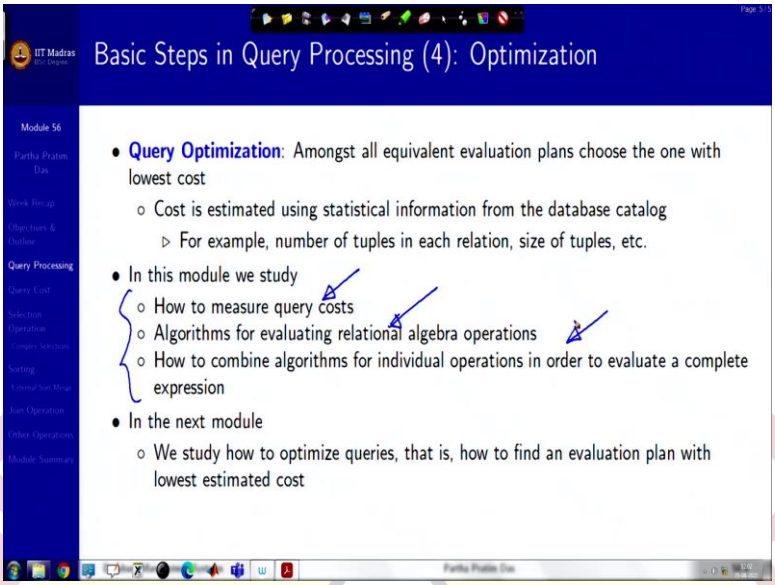
Now, just for a quick example, consider a query in SQL like this, where the relation is instructor, the condition is salary should be less than 75,000 and you want finally, the salary. So, what are the other different ways you could do this, one is you could do a direct translation of the query. For example, say from the instructor do a selection based on this condition, salary less than 75,000 and once you have got that instructor, we will have lots of different columns and then you can project it on salary to get the final result.

Or it is also possible that since salary is only attribute involved in the condition and salary is also the only attribute in the output it the other attributes of instructor actually has no role in the query. So, you could first do a select, do a projection on salary and then do the selection.

So, in the first one you do projection then selection, in the second one you do selection then projection and both of them are equivalent relational algebra expressions for the above query and we will have to see how, what are the different, what are the reasons for which one may be chosen above the other.

The relational algebra expression operations can also be evaluated through different algorithms. So, we will take some look at those algorithms and decide what could go finally, in terms of an evaluation plan we will talk more about this evaluation plan.

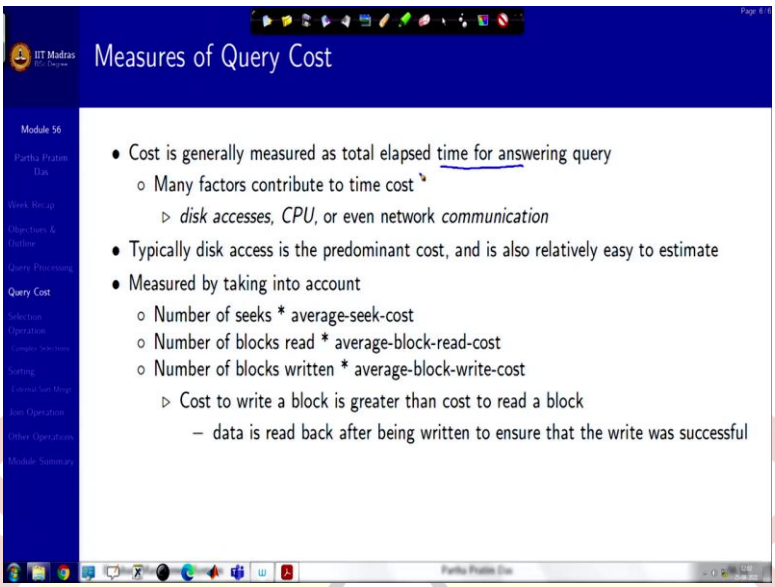
(Refer Slide Time: 06:16)

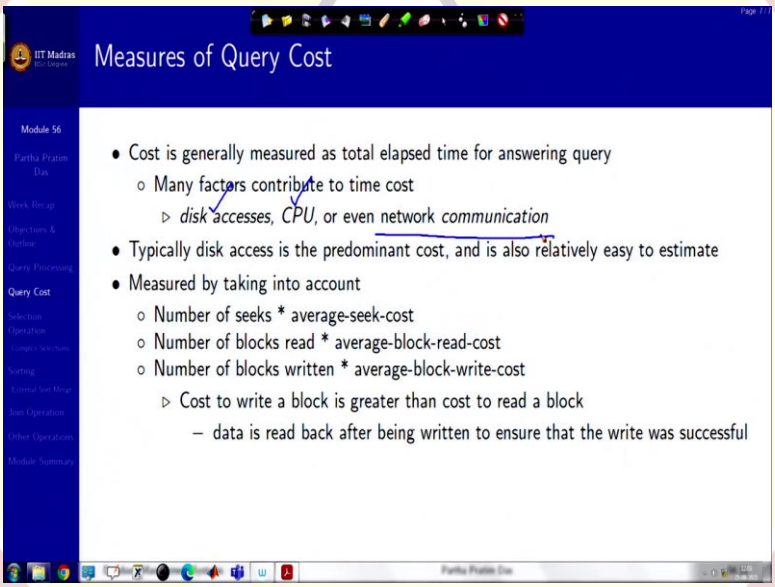


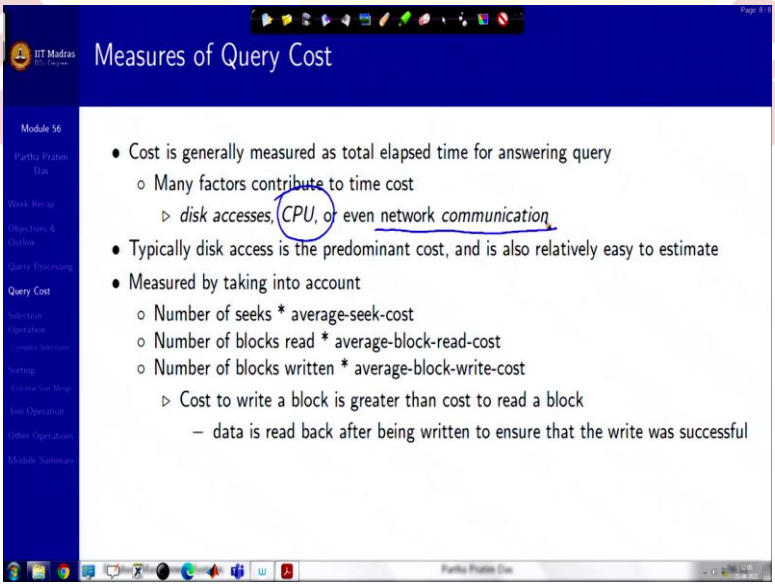
So, moving forward based on this analysis, all equivalent evaluation plans will be chosen taken and the one with the least cost will be chosen finally, for the evaluation engine and the cost is estimated as I said based on statistical information from the database like number of tuples in each relation, size of tuples, etc etc.

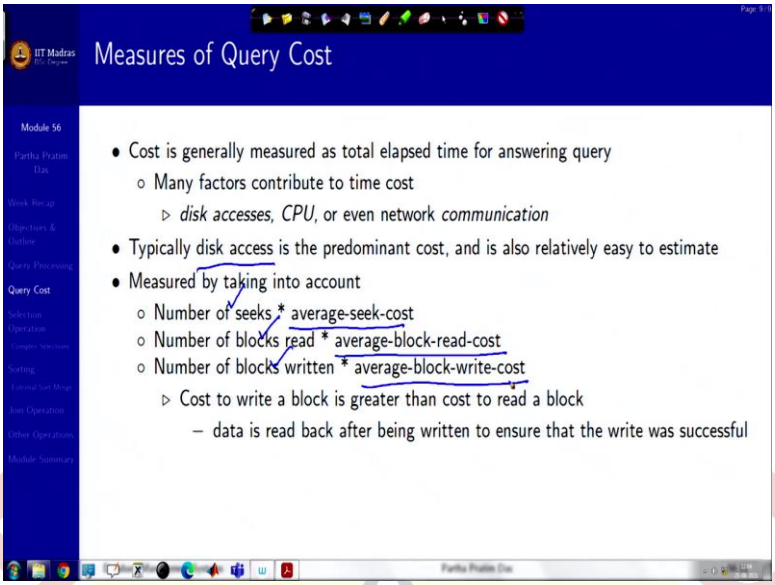
And here therefore, we need to know what this cost measures are, what are the algorithms available for evaluating relational algebra expressions and how to combine them in terms of getting a evaluation for a complete expression. So, these are the three major components, the cost, the evaluation of relation, individual relational expression, operation and then combining them into a complete expression and in the next module we talk about optimizations that are possible. So, what are the measures of cost?

(Refer Slide Time: 07:19)









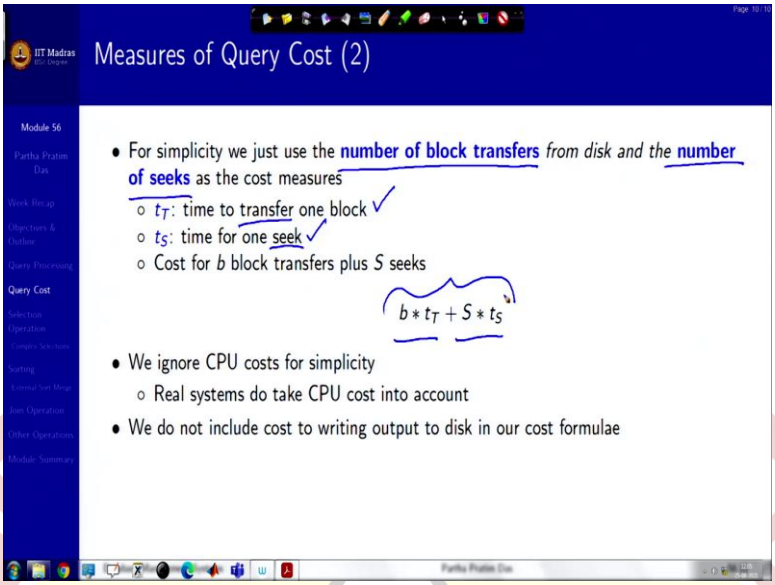
As I said the cost typically is measured in terms of total elapsed time for answering the query. So, this could be dependent on many factors, for example a primary factor is disk access, you know transferring the blocks from the disk to main memory, writing into the block back into the main memory, the time taken by the CPU for the actual execution, there may be over due to network communication, your disk may be somewhere else and your processing server could be somewhere else and there may be network delays to transfer the data and so on.

Out of these typically this CPU costs often turn out to be relatively small though we will have to in places take care consider that for our current discussion, we will ignore the considerations of communication cost, assuming that they are on a hot bus where communication has no overhead.

So, the disk access is a primary predominant cost that we are going to consider here. So, what are the components of disk access, there are we have explained that entire mechanism of parallel head movement on the disk platter system, the total spinning stack, so one part is the number of seek that you need to do, the seek will bring you to the correct track on that where you can find your block and then you will have the cost for read and the cost for write.

And you will use estimates for what is the average seek cost, average block read cost, and average block write cost, it will vary from case to case on the same system also, but we will take some average estimated value for that. Naturally the cost to write a block is far greater than the cost to read, for the simple reason that after you write you again read back to check that the write was successful. So, it will always be more than what you need for a simple read.

(Refer Slide Time: 09:27)

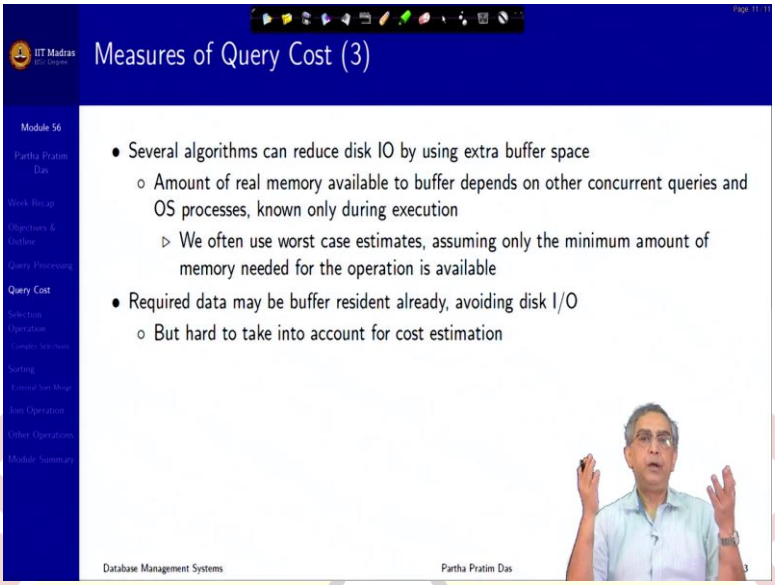


So, if we just take this in, I mean simplify this further in terms of not really looking into the read and write time, but just you know every read and write has a block transfer between the disk and the system. So, if we just take that into consideration and just take two parameters, the number of block transfers and the number of seek that happens because seek usually is a high time.

So, and take that tT is the time to transfer the block and tS is the time to seek the block, transfer time seek time. So, if there are B blocks transferred, then and s number of seek has seeking has happened, because in one seeking you may have multiple blocks being transferred. So, then the total cost is b * tT + S * tS, this is a basic expression that we will deal with.

So, we do not include the cost of writing output to the disk in the cost formula particularly, or the reading time, we will ignore those and just approximate in terms of number of block transfers.

(Refer Slide Time: 10:46)



So, given that, of course, there are several algorithms that can reduce disk I/O with using extra buffer space also, you can because as you know that the more you can buffer the less of actual accesses will have to be, will have to be performed. So, on it depends on the number of concurrent queries the ways it process, and all these, so it is very difficult to estimate it in general.

So, we typically would talk about worst case estimates. This is very similar to the way we analyse algorithms, well mostly we primarily talk of the worst-case estimate. So, with this kind of a simple cost of block transfer and sake, let us start by looking at the first most important operation that we keep on doing that is of selection, a certain condition based on which from a relation we are selecting a number of tuples, number of records.

(Refer Slide Time: 11:51)

Selection Operation: File / Index Scan

A#	Algorithm	Cost	Reason
A1	Linear Search	$t_S + b_r \times t_T$	One initial seek plus b_r block transfers
A1	Linear Search, Eq. on Key	Average case $t_S + (b_r/2) \times t_T$	Since at most one record satisfies condition, scan can be terminated as soon as the required record is found. b_r blocks transfers in worst case
A2	Prm. Index, Eq. on Key	$(h_i+1)\times(t_T+t_S)$	Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer
A3	Prm. Index, Eq. on Nonkey	$h_i \times (t_T + t_S)+ b \times t_T$	One seek for each level of the tree, one seek for the first block. Here all of b are read. These blocks are leaf blocks assumed to be stored sequentially (for a primary index) and don't require additional seeks
A4	Snd. Index, Eq. on Key	$(h_i+1)\times(t_T+t_S)$	This case is similar to primary index
A4	Snd. Index, Eq. on Nonkey	$(h_i+n)\times(t_T+t_S)$	Here, cost of index traversal is the same as for $A3$, but each record may be on a different block, requiring a seek per record. Cost is potentially very high if n is large
A5	Prm. Index, Comparison	$h_i \times (t_T + t_S)+ b \times t_T$	Identical to the case of $A3$, equality on nonkey
A6	Snd. Index, Comparison	$(h_i+n)\times(t_T+t_S)$	Identical to the case of $A4$, equality on nonkey

t_T is time to transfer one block. t_S is time for one seek
b_r denotes the number of blocks in the file
b denotes the number of blocks containing records with the specified search key
h_i denotes the height of the index. n is the number of records fetched

Selection Operation: File / Index Scan

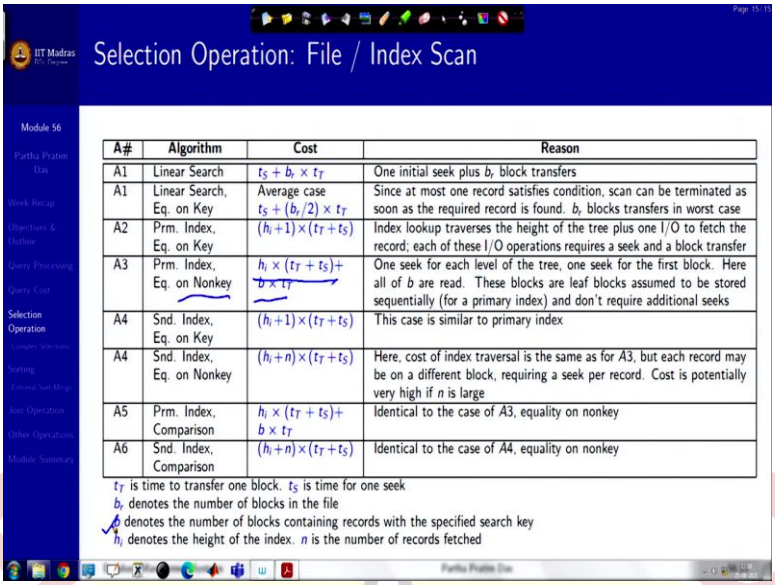
A#	Algorithm	Cost	Reason
A1	Linear Search	$t_S + b_r \times t_T$	One initial seek plus b_r block transfers
A1	Linear Search, Eq. on Key	Average case $t_S + (b_r/2) \times t_T$	Since at most one record satisfies condition, scan can be terminated as soon as the required record is found. b_r blocks transfers in worst case
A2	Prm. Index, Eq. on Key	$(h_i+1)\times(t_T+t_S)$	Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer
A3	Prm. Index, Eq. on Nonkey	$h_i \times (t_T + t_S)+ b \times t_T$	One seek for each level of the tree, one seek for the first block. Here all of b are read. These blocks are leaf blocks assumed to be stored sequentially (for a primary index) and don't require additional seeks
A4	Snd. Index, Eq. on Key	$(h_i+1)\times(t_T+t_S)$	This case is similar to primary index
A4	Snd. Index, Eq. on Nonkey	$(h_i+n)\times(t_T+t_S)$	Here, cost of index traversal is the same as for $A3$, but each record may be on a different block, requiring a seek per record. Cost is potentially very high if n is large
A5	Prm. Index, Comparison	$h_i \times (t_T + t_S)+ b \times t_T$	Identical to the case of $A3$, equality on nonkey
A6	Snd. Index, Comparison	$(h_i+n)\times(t_T+t_S)$	Identical to the case of $A4$, equality on nonkey

t_T is time to transfer one block. t_S is time for one seek
b_r denotes the number of blocks in the file
b denotes the number of blocks containing records with the specified search key
h_i denotes the height of the index. n is the number of records fetched

Selection Operation: File / Index Scan

A#	Algorithm	Cost	Reason
A1	Linear Search	$t_S + b_r \times t_T$	One initial seek plus b_r block transfers
A1	Linear Search, Eq. on Key	Average case $t_S + (b_r/2) \times t_T$	Since at most one record satisfies condition, scan can be terminated as soon as the required record is found. b_r blocks transfers in worst case
A2	Prm. Index, Eq. on Key	$(h_i+1)\times(t_T+t_S)$	Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer
A3	Prm. Index, Eq. on Nonkey	$h_i \times (t_T + t_S)+ b \times t_T$	One seek for each level of the tree, one seek for the first block. Here all of b are read. These blocks are leaf blocks assumed to be stored sequentially (for a primary index) and don't require additional seeks
A4	Snd. Index, Eq. on Key	$(h_i+1)\times(t_T+t_S)$	This case is similar to primary index
A4	Snd. Index, Eq. on Nonkey	$(h_i+n)\times(t_T+t_S)$	Here, cost of index traversal is the same as for $A3$, but each record may be on a different block, requiring a seek per record. Cost is potentially very high if n is large
A5	Prm. Index, Comparison	$h_i \times (t_T + t_S)+ b \times t_T$	Identical to the case of $A3$, equality on nonkey
A6	Snd. Index, Comparison	$(h_i+n)\times(t_T+t_S)$	Identical to the case of $A4$, equality on nonkey

t_T is time to transfer one block. t_S is time for one seek
b_r denotes the number of blocks in the file
b denotes the number of blocks containing records with the specified search key
h_i denotes the height of the index. n is the number of records fetched



So, here are a present different scenarios that are possible under the name of A 1, A 2 like that, and the algorithm that it involves, for example you can use a simple linear search, search through the entire file, linear search. So, what will it involve, it will involve an initial seek and then the block transfers where, at the bottom of the table you will see the legend given b r is the number of blocks in the file. So, it will reach there keep on transferring all the blocks.

So, it is this is the transfer time, this is a seek time and the linear search will need to this. Now, if you do a linear search, but you are just looking for equality on key, you have a key and you were looking for equality on key which is very often but you do filtering on the key. So, what will happen your, in one record, once a record satisfies this condition your search will have to stop, because key is unique. So, you can have only one record in the output.

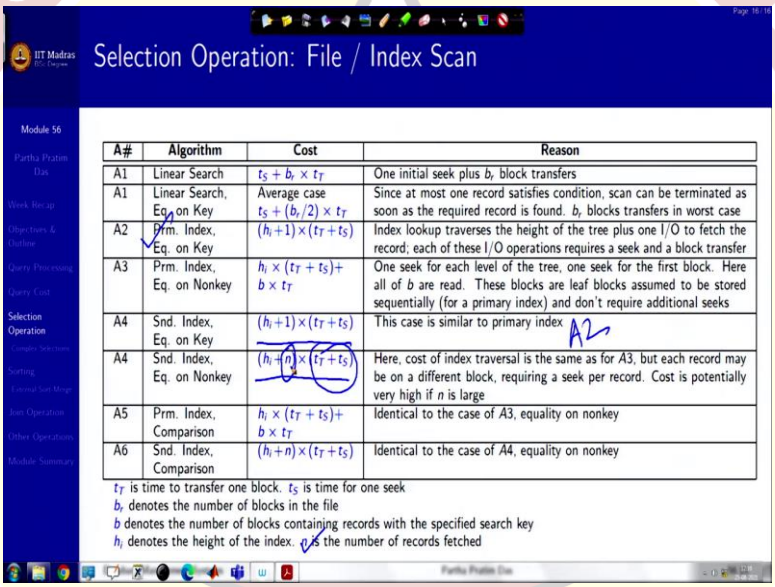
So, what it means that on the average you can assume that half the blocks will always be searched in a general case. So, here you have a reduction in the number of transfers that you need to do, though in the worst case it will always, it will remain same as the previous one. Now, let us suppose that you have using a primary index on the key and you are looking for equality, if you are looking for primary index, then it necessarily depends on the height of the B plus tree at that point of time hi.

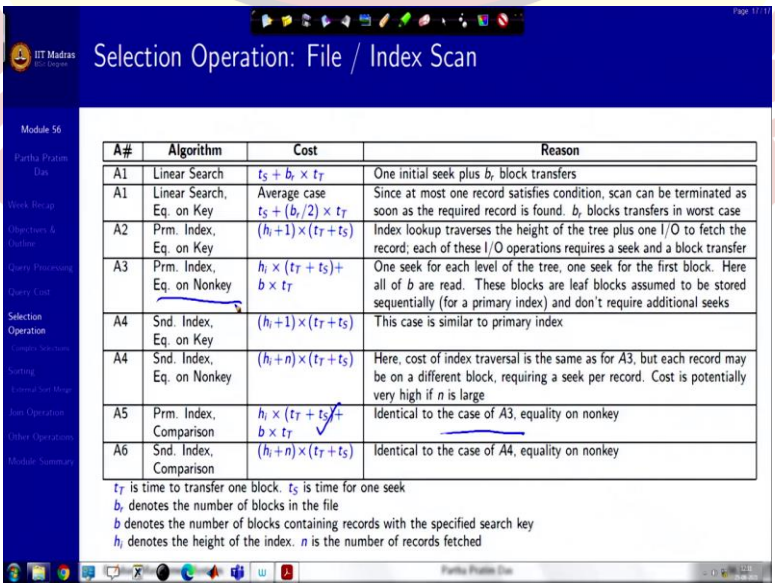
So, it is possible that you will have to go, you will have to go down to the leaf node, actually will always have to go down to the leaf node and then do an additional operation to get the final record values. So, when you do that, so you have hi + 1 number of operations to be done, every operation since you are using an index, they are not going to consecutive places, they are going at different places. So, every operation will need a seek and transfer time.

So, this is why you get this kind of a time. You would expect this to be much less than what you find in the linear search because h will be significantly small, maybe 4 or 5, 10 like that. But if you do that on a non-key, if you are doing it on a non-key then you do not have that so easy.

You will again have to go over the height for the entire B plus tree, doing the seek as well as the transfer. But finally given the records, the number of blocks which have your final record, not b r b, this is not the number of blocks in the file, this is number of blocks which contain the search key and this is non-key, so there could be many. So, all of those will have to be read. So, many transfers will be required along with that.

(Refer Slide Time: 15:09)



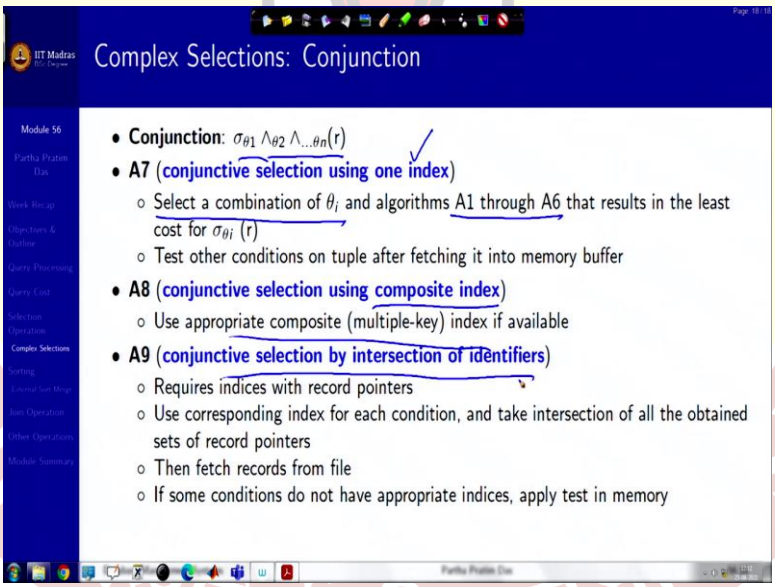


So, if you have a secondary key secondary index on the key it is same like A 2, it same like A 2, of course, because it is an index anyway, if you have a secondary index on a non-key, then your cost will depend on, how many records you are actually fetching, because it is a non-key, so it is distributed everywhere.

So, it is hi + n and this will make naturally that cost very, very high if n is large, you can see that because you are having to multiply each, for each record, you will eventually might have a transfer and seek cost. For primary index using primary index, when you look for comparison, you will the case is very similar to working on A 3 that is primary index on non-key because it is one additional operation for that.

And similarly, if you are using a secondary index, it is similar to A 4. So, these are different, you know simple selection operations and given the possibility of index or whether we area querying for a key or a non-key or comparison, you will have these kind of different cost situations that you can use as estimates of the query.

(Refer Slide Time: 16:34)



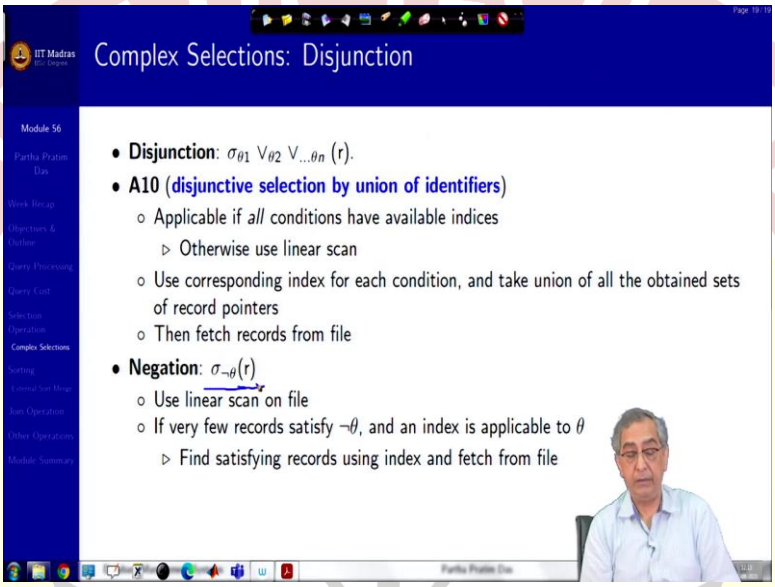
Of course, there could be more complex selections often there are because you have conjunctions in the query, that is all these conditions theta 1 to theta n are actually added. So, you can select a combination of theta i and corresponding algorithms A 1 to A 6 as we have them, that result in the best cost for this theta i and test other conditions on tuple fetching it into the memory.

So, you can since you have one index, so here is the case is you have one index so, you are using that index to make one particular condition to be satisfied faster and those are a, that is

subset of the relation and then you apply other conditions in the memory buffer possibly sizes become less.

If it is a composite index, then you have to use appropriate multi key index, if you have conjunction by conducting selection by intersection of identifiers that is there are some require indices with the record pointers you have to use the corresponding index and so on. So, it gets more and more clumsy and complicated.

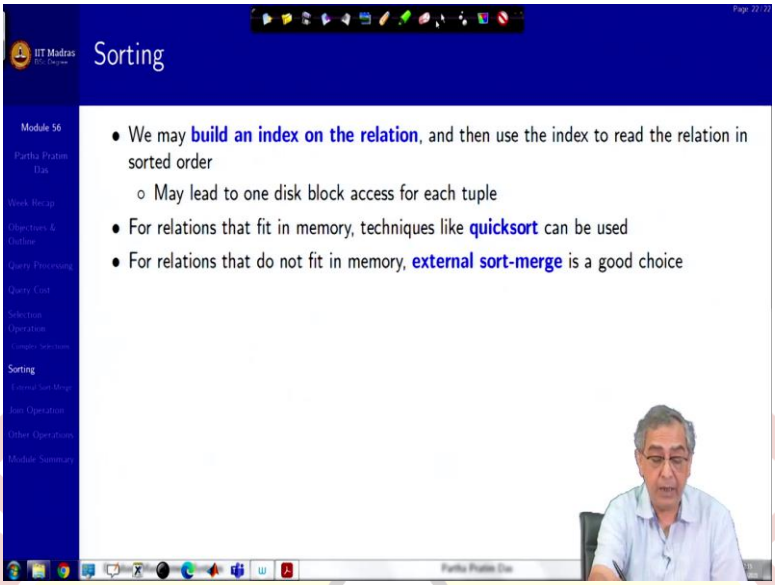
(Refer Slide Time: 17:55)



Similar case you have for disjunction, since available all conditions have available indices. If we have that, then we can use those, but typically it will not be the case. So, you will typically do a linear scan because, you do not know which can be because you cannot filter anything, earlier when you were using conjunction, if you satisfy on 1 theta i then you know that the result is a subset of that, but here if you apply 1 theta i the result is a superset of that.

So, you cannot. So, you have to do the, use the indexes that are available take union of the obtained sets and get the result similar thing will hold for negation, you will have to do a linear scan and get negations. So, these are like what you do for selection.

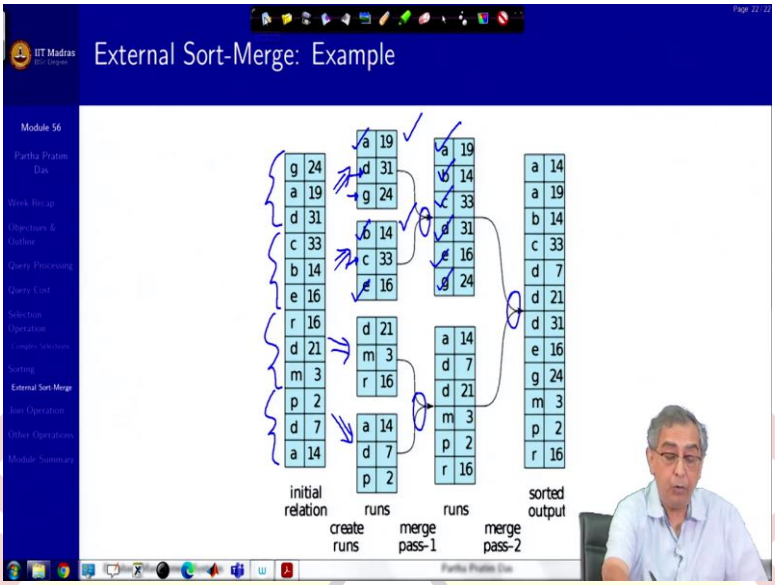
(Refer Slide Time: 18:59)



Let us look at the other options sorting which will often need. Now sorting is can be done in multiple ways one is you can build an index on the relation and then use the index to read the relation in sorted order. But they this may lead to a very bad block access pattern, for each tuple, because they may not come in a consecutive way, one comes here, one comes there, one comes here, one comes there like that, it will go on.

So, what is preferred is if the relation is small to fit into the memory bring it to the memory do a something like a quicksort, which is an in-memory algorithm. If it does not, then you will have to do a multi way external sort and merge, merge sought external merge. So, that is you are not being able to fit the entire thing in memory. So, some parts are will have to be brought into the memory sorted there by some algorithm and then written back to the disk and other parts you bring and keep on merging that.

(Refer Slide Time: 20:02)



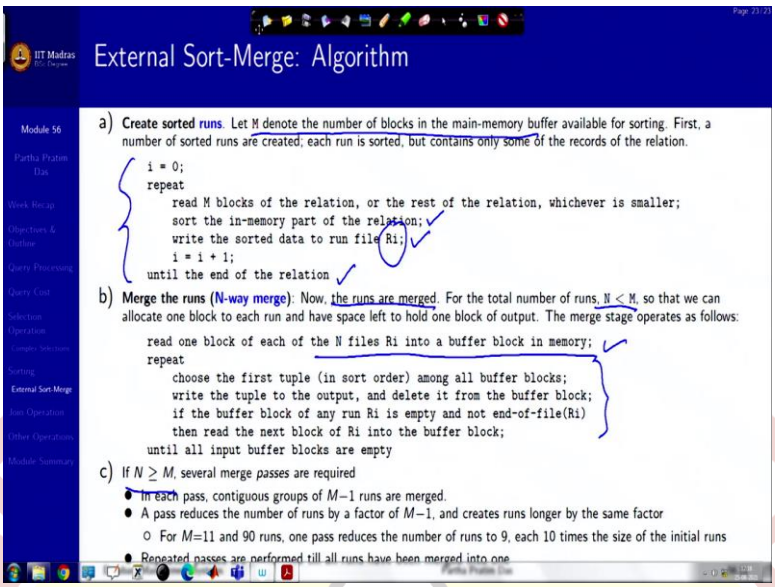
So, the basic strategy for that is sort and merge. So, for example, here I mean, this is just a variant of the merge sort, but because it is external there is a cost of sorting in memory is much less here compared to the block transfers you need to do, to keep back the partial result this becomes a different kind of an algorithm, but grossly the same.

So, what you do is you basically divide the original relation in terms of runs of certain size say runs of 3 which can fit into the memory. So, you bring each in the memory and sort each one of them, this is sorted here, this is sorted here, this is sorted here and this is sorted here. So, these are the runs initially and then what you do is take 2 runs and merge them.

So, a you know this is a smallest in this run, this is a smallest in this run. So, you put a smaller than the 2, then this is the smallest d between d and b you put b, then this is the smallest here this is the smallest here between d and c you put c, then you put d, then you will have e and so, on. So, you merge them.

So, as you do that, you will also need, may need to do it in parts for bringing in the memory and doing that merge. Similarly, you merge this then you merge the entire result and in this process, so we will keep on going bottom up like this in the merge sort.

(Refer Slide Time: 21:44)



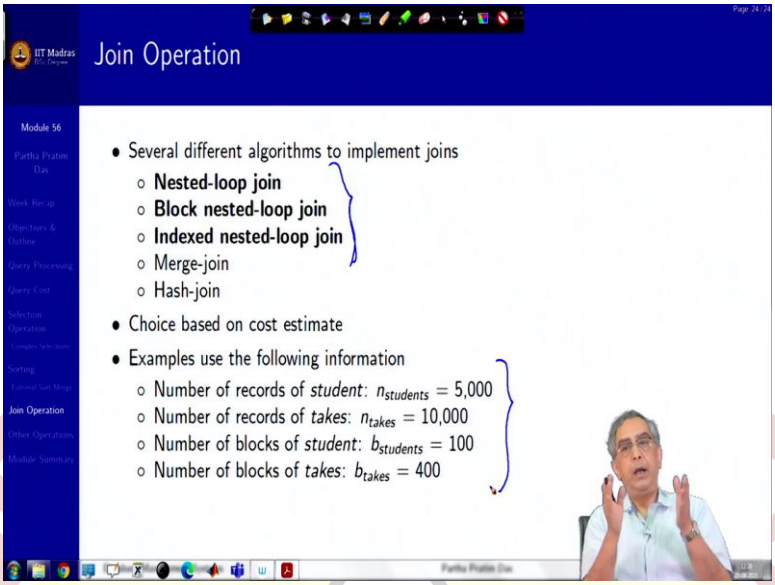
So, here is a basic outline of the algorithm the first stage, which is creating the run, which if you have say, number of blocks that fit into the main memory buffer if that is n, then m, then every time you read m and or the rest of the relation whichever is smaller and sort the in memory part and write the sorted data to run file R i and keep on in incrementing i.

So, you will have so many like we had 4 run files. So, this will continue till you have the relation exhausted is the maximum that you could have created in terms of the runs. Next the merging has to happen. So, sort merge, the sort part is done, this is the merging has to happen. So, there are two possibilities one is the number of runs could be less than m, the total number of rounds n could be less than m, the number of blocks that fit into the main memory.

So, we can allocate one block to each run and have the space left to hold the block of the output. So, you keep on doing this read one block, each of the n files R i that you have created here, the runs and into the buffer in the memory and then you do a basic merging till it continues and then you write it back.

So, this will continue till all input buffer blocks are empty, very clear. If it is, if n is not less than m, that is there are more runs than the number of blocks available, then you will again have to do it in groups and do multiple parses through them. So that is a simple, common-sense extension and that is the way you can sort externally efficiently any large relation it is going to be very time taking costly, but in case you need to do that, you would have to do this.

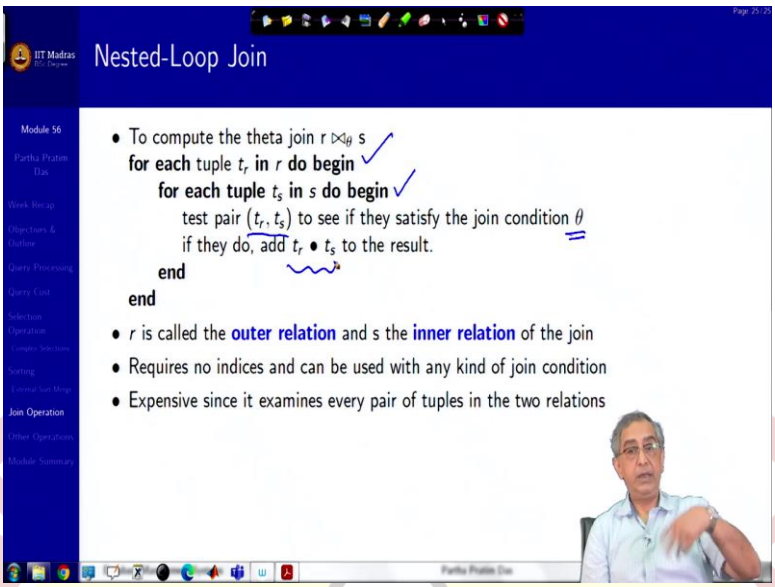
(Refer Slide Time: 23:53)



The probably besides selection, one of the most important operation that we have talked about is join. So, when we join two relations, we basically do a Cartesian product and then we do some specific selection based on that whether it is a condition or it is equality, natural join and so on.

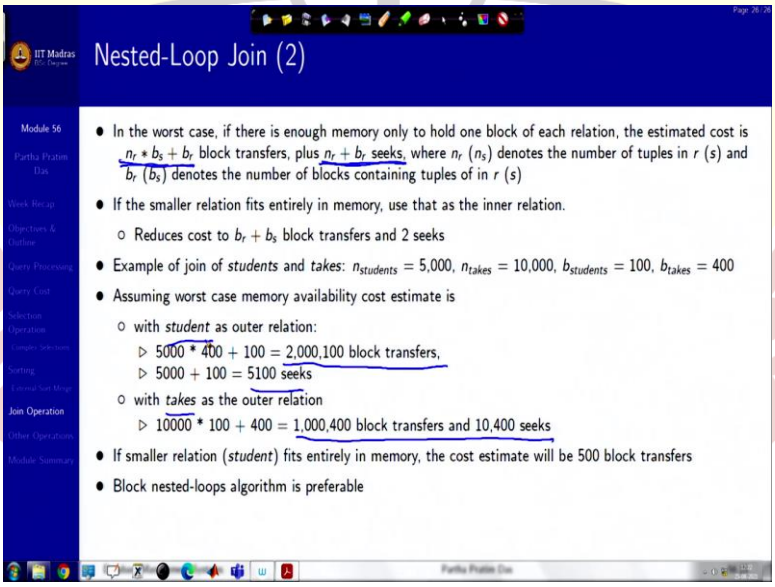
So, there are different ways of doing that and these three are what we are going to discuss here, there are some more as you can see and we will use this as an example to illustrate that there are two records to fire relations students, which has 5000 records and 100 blocks for that and another is takes, student takes courses, so takes which has student role number possibly 2 as a joint field and which has 10,000 records and has 400 blocks, the number of blocks corresponding to the number of records will vary, because the records size is different between students and takes.

(Refer Slide Time: 25:05)



Given this, if we look at the nested loop join is a easiest that what you do is for each tuple in the outer relation say r the first one, you take the tuple from the inner relation and test if this pair satisfy the condition theta, if they satisfy the condition theta, then concatenate them and put it to the result otherwise you continue.

(Refer Slide Time: 25:35)

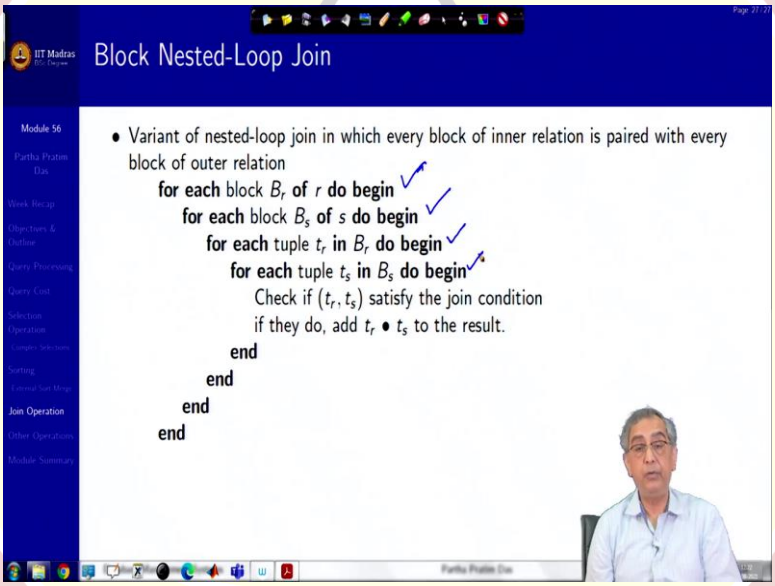


So, what will this mean, this naturally for every tuple in the outer relation, you will have to get the entire of the inner relation, you have to access the entire of the inner relation because we are going one by one. So, that will mean that you have so many seeks and so, many block transfers. So, many block transfers overall and so, many seek that will happen.

So, this is going to be quite some expensive operation and if you take the numerical this is the number of block transfers, if you use student as the outer relation and this is the number of block transfers and seeks if you use tasks as the outer relation. So, you can see that actually it is immaterial as to in which order we do this join r theta join s or s theta join r will always give you the same result except for the final ordering of the attributes.

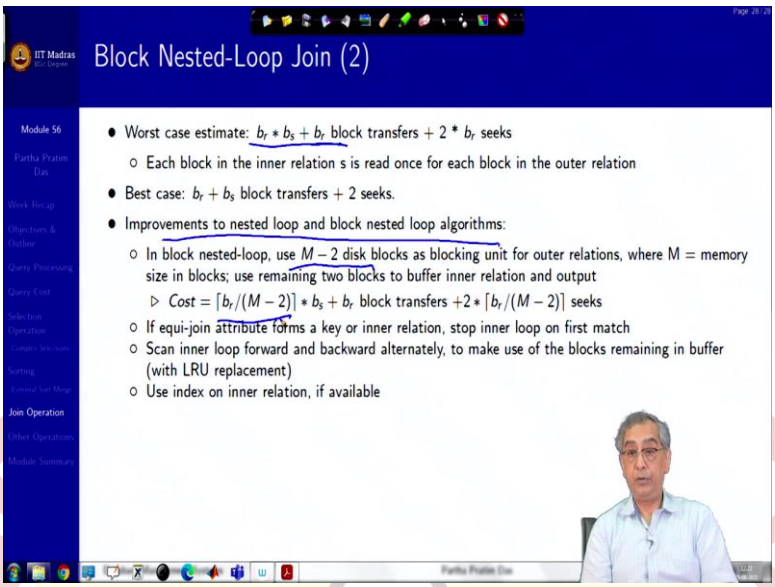
But what is important is you can see because of the fact that the inner relation needs to be read all the time. So, the smaller the relation it should occur inside. So, since a takes is a larger relation, so it will be, if you keep it inside as an inner relation, then naturally so many more times you will have to get the tuples of takes. So, that is the reason that if you take keep take as a outer relation, you get much less number of block transfers required.

(Refer Slide Time: 27:20)



So, you can improve on that by not checking on every tuple because you cannot finally read a tuple, you read a block. So, you simply do this for each block of the relation b r relation r, then each block of the relation s and then once you have got this block you do the tuple base. So, this entire thing now is happening in memory. So, it gets reduced to the number of blocks that you have to do.

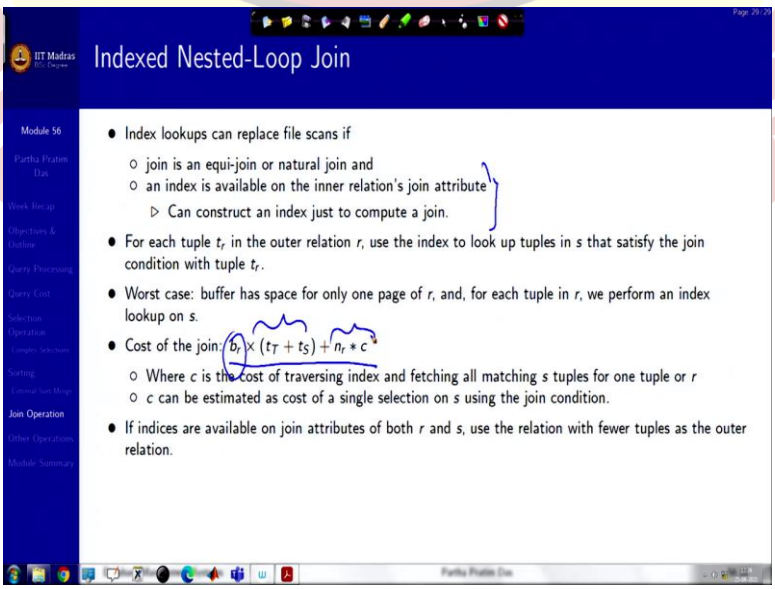
(Refer Slide Time: 27:51)



And therefore, your cost drastically goes down to br * b s + b r, this is quite understandable, b r * bs because the smaller one will have to be done every time you have to bring the blocks of the smaller relation in a relation, not the smaller relation the inner relation as many times as there are blocks in the outer relation.

So, if you can, you can also do some improvements into this because in the block nested loop, you may use M - 2 disk blocks keeping 2 for the output, you can do M - 2, disk blocks at a single goal. So, if you do that, then this goes down by a factor of M - 2. So, which will give you a drastically better cost.

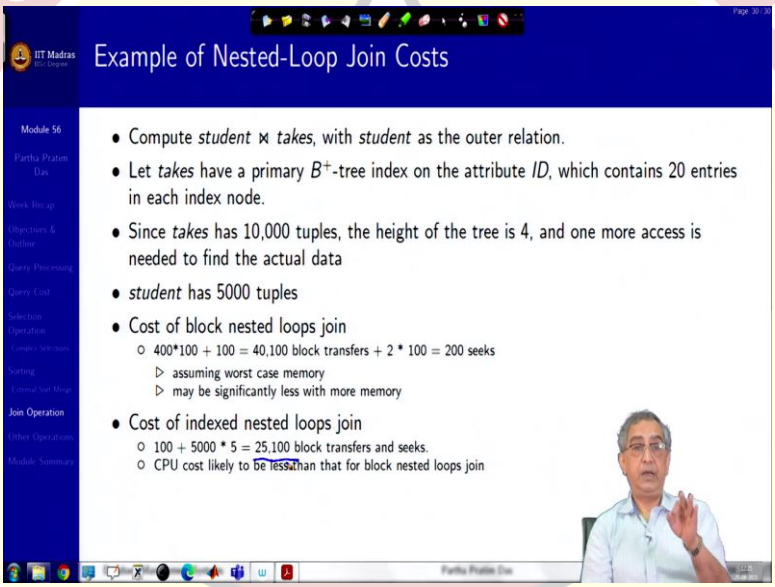
(Refer Slide Time: 28:55)



What will be even better is if you have an index on the file and particularly if you are doing equijoin or natural join, which is where you have an index available on the inner relations join attribute. So, that means if you have in a relation join attribute is the most frequently accessed. So, if you have an index on that, then you will have a join which is, which will go much faster than the block-based nesting.

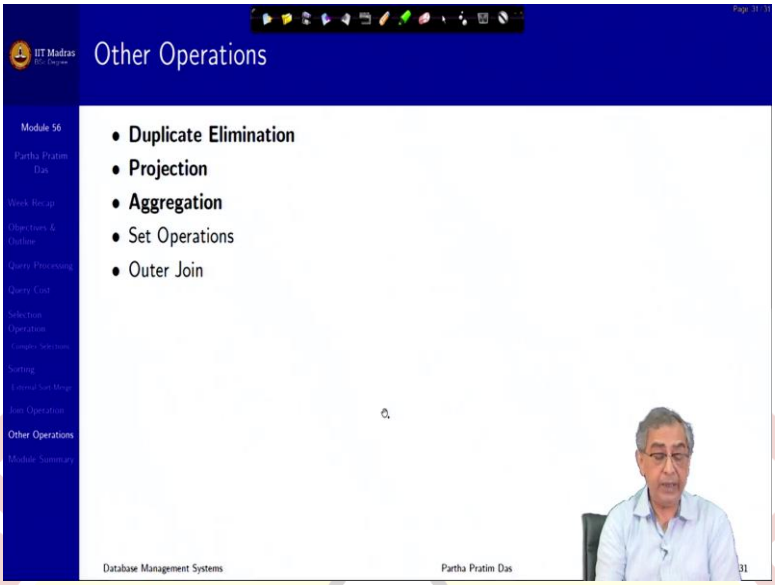
And you can show that this comes down to be br times because br the outer number of blocks you will have to read and then nr is number of records, number of tuples in that and see basically is the cost to get it through the index, for a single selection. So, this will be even more fast.

(Refer Slide Time: 29:57)



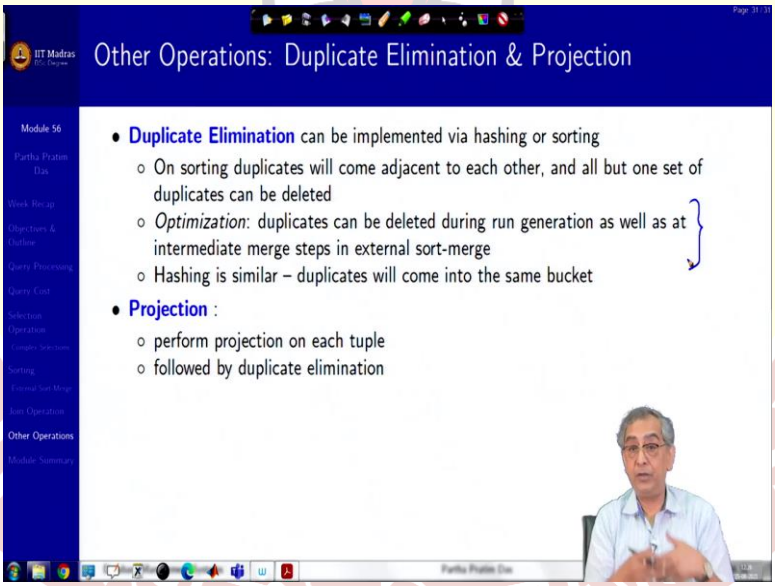
So, here I have just using the same data, just I have just shown that how this now gets drastically reduced particularly it was some 2 million or something it has gone down to 25,000 if you have used the proper index for the nested loop join. So that is the kind of clue to you know, how you handle join.

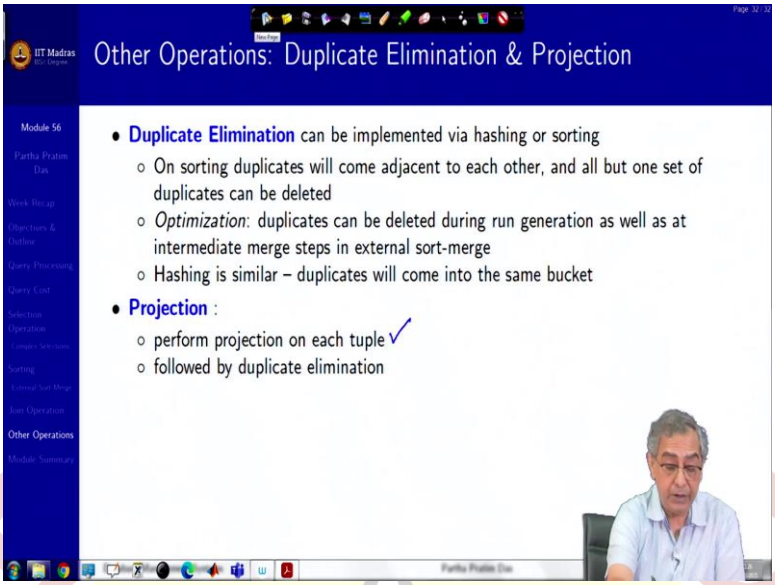
(Refer Slide Time: 30:25)



There are several other operations for example, duplicate elimination, projection, aggregation and so on.

(Refer Slide Time: 30:30)

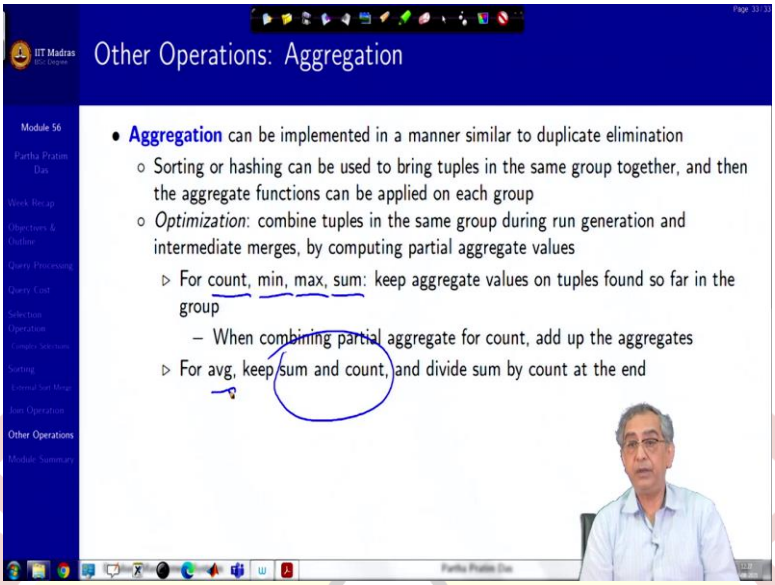




So, how do you eliminate duplicate the way is to either do sorting, if you do sorting, then the duplicates will come adjacent to each other and you can just keep one and remove the rest or you can do hashing, where they will go to the same bucket keep one and remove the rest, but the optimization that you can do is in sorting as we have seen, you are doing an external sort merge typically.

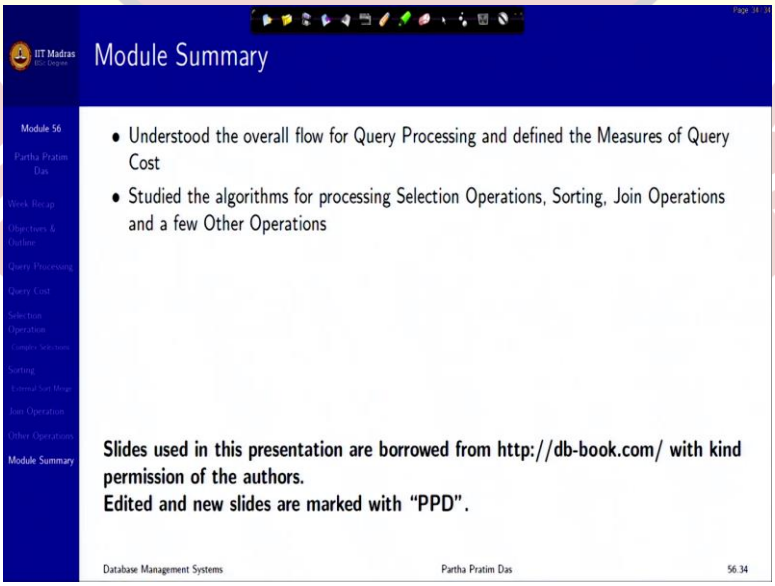
So, when you are doing that, when you are merging, you can get to see the duplicate. So, drop them right there in the intermediate stage itself. So, when you saw in the original file, there are duplicates, when you actually have sorted the duplicates will not appear. So, do it on the fly, which makes it more efficient, because if there are too many duplicates, they will very soon go down similar way you can do perform projections on each tuple also and follow it by duplicate elimination, because that is what will regularly happen when you do projection.

(Refer Slide Time: 31:29)



When you have to compute aggregation, you can, similar way to duplicate elimination, you can collect all the tuples which satisfy the aggregation group and then compute the aggregate, but you can optimise by while you are collecting tuples of the same group at that time keep on computing the aggregate on the fly like count, min, max, sum these all can be computed in that manner except average, because it depends on the whole thing. So, to compute average, you actually compute sum and count and at the end you divide by giving the average, so this way, you can certainly optimise and get faster operations.

(Refer Slide Time: 32:14)



So, we in this module, we took a quick look into the query processing flow and the typical costs and some of the query processing strategies of very common operations that you have

in relational algebra, will stop here. Thank you very much for your attention. We will continue and discuss about optimization in the next module.